

AULA 7

Autenticação com JWT

Disciplina: CK0207 — Desenvolvimento de Software para Web

Professor: Dr. Fernando Antônio Mota Trinta

Universidade Federal do Ceará (UFC)

Projeto do curso: `nest-api-curso` (continuação da Aula 6)

Autor: Samuel Sales Furtado

Objetivos de Aprendizagem

- Diferenciar autenticação de autorização
- Entender o que é um JWT e como ele viaja numa requisição
- Fazer hash de senhas com bcrypt, sem nunca guardar texto puro
- Entender o papel do Passport e das Strategies no NestJS
- Entender o papel de um Guard na proteção de rotas

Esta é uma aula conceitual: instalamos as dependências e escrevemos a `JwtStrategy`, mas os endpoints de registro/login de verdade e o guard aplicado numa rota ficam para a **Aula 8** (Laboratório: Proteção de Rotas).

1. Autenticação vs. Autorização

Autenticação responde "quem é você?" — o processo de verificar uma identidade, normalmente com login e senha, checando se as credenciais batem com um usuário cadastrado. **Autorização** responde "o que você pode fazer?" — só depois de saber quem é o usuário é que faz sentido decidir se ele tem permissão para aquela ação ou recurso específico.

Exemplo prático: numa rota `DELETE /users/:id`, a autenticação garante que quem faz a requisição é quem diz ser. Isso, sozinho, não decide se **esse** usuário pode remover **aquele** registro específico — essa segunda decisão é autorização, e vem depois da autenticação, nunca no lugar dela.

O NestJS **não vem com autenticação embutida**. Ele se integra com o **Passport** — a biblioteca de autenticação mais usada no ecossistema Node — através do pacote `@nestjs/passport`. O Passport funciona com **strategies**: cada uma sabe validar um tipo de credencial (usuário/senha, JWT, OAuth...). Este curso usa a strategy de JWT, *token-based*: o servidor não guarda estado de "quem está logado" — toda informação necessária viaja dentro do próprio token.

2. Hash de Senhas com bcrypt

Nunca armazene senha em texto puro. O `bcrypt` gera um hash unidirecional, com salt embutido, a partir da senha original.

```
const hash = await bcrypt.hash(senhaDigitada, 10); // 10 = rounds (custo)
const bateu = await bcrypt.compare(senhaDigitada, hashSalvoNoBanco);
```

`hash()` gera o hash na hora do cadastro — o número de rounds é o custo computacional: quanto maior, mais difícil de quebrar por força bruta, mas mais lento de calcular. `compare()` verifica, no login, se a senha digitada — quando hasheada — bate com o hash guardado, sem nunca reverter o hash de volta à senha original (o que, matematicamente, não é possível).

3. JWT — o Fluxo do Token

1. Cliente envia usuário/senha em `POST /auth/login`.
2. Servidor valida com `bcrypt.compare()` e, se baterem, assina um JWT com uma chave secreta.
3. Cliente guarda o token e envia `Authorization: Bearer <token>` nas próximas requisições.
4. Servidor valida a assinatura a cada requisição — **sem precisar consultar o banco toda vez**.

Esse último ponto é o que torna JWT popular para APIs: escala bem, porque o servidor não mantém uma tabela de sessões ativas. Uma limitação a considerar: revogar um token antes da hora normalmente exige estratégia extra (lista de tokens invalidados, ou tempo de expiração curto) — tópico para aprofundar além deste curso.

4. Passport e Guards

```

@Injectable()
export class JwtStrategy extends PassportStrategy(Strategy) {
  constructor() {
    super({
      jwtFromRequest: ExtractJwt.fromAuthHeaderAsBearerToken(),
      ignoreExpiration: false,
      secretOrKey: process.env.JWT_SECRET,
    });
  }

  async validate(payload: { sub: number; email: string }) {
    return { userId: payload.sub, email: payload.email };
  }
}

```

`jwtFromRequest` diz de onde extrair o token (o cabeçalho `Authorization`, como Bearer). `secretOrKey` é a mesma chave usada para assinar o token no login, agora usada para validar a assinatura. `validate()` roda depois que o Passport já confirmou a assinatura — o que ele retornar vira `request.user`.

```

@UseGuards(JwtAuthGuard)
@Get('me')
getProfile(@Request() req) {
  return req.user;
}

```

O **Guard** roda antes do handler do controller: sem token válido, o Nest já devolve `401 Unauthorized` sozinho — nenhuma linha do controller é executada. Guard e Strategy têm responsabilidades isoladas, o mesmo princípio de separação seguido desde a Aula 1.

Laboratório — Passo a Passo ## Passo 1 — Instalar as dependências

```

npm install @nestjs/passport passport @nestjs/jwt passport-jwt bcrypt
npm install --save-dev @types/passport-jwt @types/bcrypt

```

Passo 2 — Demo rápida do bcrypt (opcional)

```

import * as bcrypt from 'bcrypt';

async function demo() {
  const hash = await bcrypt.hash('minhaSenha123', 10);
  console.log('Hash gerado:', hash);
  console.log('Senha certa bate?', await bcrypt.compare('minhaSenha123', hash));
  console.log('Senha errada bate?', await bcrypt.compare('senhaErrada', hash));
}

demo();

```

Testar: rodar e mostrar que o hash nunca se repete, mas `compare` sempre reconhece a senha certa.

Passo 3 — Gerar o módulo de autenticação

```

nest g module auth
nest g service auth
nest g controller auth

```

Passo 4 — Criar a `JwtStrategy` e registrar no `AuthModule` Usar o código da Seção 4. No `AuthModule`:

```
@Module({
  imports: [
    PassportModule,
    JwtModule.register({
      secret: process.env.JWT_SECRET || 'segredo-temporario-trocar-depois',
      signOptions: { expiresIn: '1h' },
    }),
  ],
  controllers: [AuthController],
  providers: [AuthService, JwtStrategy],
})
export class AuthModule {}
```

Importar no `AppModule`: `imports: [PrismaModule, UsersModule, AuthModule]`.

Testar: confirmar que o servidor sobe sem erro. Ainda não há rota nova para testar via curl — isso vem na Aula 8.

O que ainda falta (gancho para a Aula 8)

- Implementar `AuthService.validateUser()` e `AuthService.login()`.
- Criar os endpoints `POST /auth/register` e `POST /auth/login`.
- Criar o `JwtAuthGuard` e aplicar `@UseGuards(JwtAuthGuard)` numa rota real.
- Mover o `JWT_SECRET` para o `.env`, carregado via `dotenv/config`.
- Adicionar o campo `password` ao model `User` (nova migration) e garantir que ele nunca seja devolvido nas respostas da API.

Verificação de Conformidade

- ✔ Ausência de autenticação embutida no Nest, integração via Passport — conferido em docs.nestjs.com/security/authentication.
- ✔ `bcrypt.hash()` / `bcrypt.compare()` — conferido na documentação oficial do pacote `bcrypt`.
- ✔ `JwtStrategy extends PassportStrategy(Strategy)`, `jwtFromRequest`, `secretOrKey` — conferido em docs.nestjs.com/security/authentication.
- ✔ `@UseGuards(JwtAuthGuard)` devolvendo 401 automaticamente — conferido em docs.nestjs.com/guards.
- ⚠ O valor `'segredo-temporario-trocar-depois'` é só um placeholder para esta aula conceitual — nunca usar em produção.